MapReduce Tasks

Initial Setup and Common Commands:

1. Loading csv files into input_dataset. The four CSV files which are used for Task1,2,and 3 are used for all the Map Reduce Tasks.
2. Copying the csv files to HDFS directory 'yellow_taxi'
3. Create python files for mrtask (mrtask_xx.py) using 'nano'
4. Making the python file executable using chmod.

   chmod +x mrtask_xx.py

5. Run the Hadoop streaming job using the command to get the output of the task and store the output in the folder '***/user/Hadoop/mrtask_xx***'.

   hadoop jar /usr/lib/hadoop/hadoop-streaming.jar -D mapreduce.job.reduces=1 -files mrtask_xx.py -mapper "python3 mrtask_xx.py mapper" -reducer "python3 mrtask_xx.py reducer" -input /user/hadoop/yellow_taxi -output /user/hadoop/mrtask_xx

6. Check the output using the following command.

   hadoop fs -cat /user/hadoop/mrtask_xx/part-*

Task A

Which vendors have the most trips, and what is the total revenue generated by that vendor?

Code:

```
#!/usr/bin/env python3

import sys
from datetime import datetime

# Mapper
def mapper():
  # Skip header
  header = True

  for line in sys.stdin:
    if header:
      header = False
      continue

    # Remove leading/trailing whitespace
    line = line.strip()

    # Split the line into fields
    fields = line.split(',')

    try:
```

```
        if len(fields) >= 17:  # Ensure we have enough fields
            vendor_id = fields[0]
            total_amount = float(fields[16])

            # Emit key-value pair: vendor_id \t 1,total_amount
            print(f'{vendor_id}\t1,{total_amount}')

    except ValueError:
        # Skip lines with invalid data
        continue

# Reducer
def reducer():
    current_vendor = None
    trip_count = 0
    total_revenue = 0.0

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        # Parse input from mapper
        vendor_id, value = line.split('\t')
        count, amount = value.split(',')

        try:
            count = int(count)
            amount = float(amount)
        except ValueError:
            continue

        # If this is the first vendor or same vendor, accumulate
        if current_vendor == vendor_id:
            trip_count += count
            total_revenue += amount
        else:
            # If we have a previous vendor, output their results
            if current_vendor:
                print(f'{current_vendor}\t{trip_count},{total_revenue:.2f}')

            # Start accumulating for new vendor
            current_vendor = vendor_id
            trip_count = count
            total_revenue = amount

    # Output the last vendor
    if current_vendor:
        print(f'{current_vendor}\t{trip_count},{total_revenue:.2f}')

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```

The output is in the form - Vendor, Trip count and Total Revenue.

```
hadoop@ip-172-31-76-104:~
                Reduce input groups=2
                Reduce shuffle bytes=148311975
                Reduce input records=39223146
                Reduce output records=2
                Spilled Records=78446292
                Shuffled Maps =87
                Failed Shuffles=0
                Merged Map outputs=87
                GC time elapsed (ms)=4102
                CPU time spent (ms)=231870
                Physical memory (bytes) snapshot=15115186176
                Virtual memory (bytes) snapshot=109227237376
                Total committed heap usage (bytes)=15619588096
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=3695313456
        File Output Format Counters
                Bytes Written=48
25/01/13 04:28:35 INFO streaming.StreamJob: Output directory: /user/hadoop/mrtask_a
[hadoop@ip-172-31-76-104 ~]$ hadoop fs -cat /user/hadoop/mrtask_a/part-*
2       21376457,343476761.39
1       17846689,281982638.93
[hadoop@ip-172-31-76-104 ~]$ nano mrtask_a.py
[hadoop@ip-172-31-76-104 ~]$
```

Answer:

Vendor 2 (Verifone Inc.) had the most number of trips 21376457 and the total revenue generated is 343476761.39

Task B

Which pickup location generates the most revenue?

Code:

```
#!/usr/bin/env python3

import sys

def mapper():
  # Skip header
  header = True

  for line in sys.stdin:
    if header:
      header = False
      continue

    # Remove leading/trailing whitespace
    line = line.strip()

    # Split the line into fields
    fields = line.split(',')

    try:
      if len(fields) >= 17:  # Ensure we have enough fields
        pickup_location = fields[7]   # PULocationID
        total_amount = float(fields[16])  # total_amount

        if total_amount >= 0:  # Only consider non-negative amounts
          # Emit: pickup_location \t total_amount,1
          print(f'{pickup_location}\t{total_amount},1')

    except (ValueError, IndexError):
```

```
        # Skip lines with invalid data
        continue

def reducer():
    current_location = None
    total_revenue = 0.0
    trip_count = 0

    # Store results to sort later
    results = []

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        try:
            # Parse input from mapper
            location, value = line.split('\t')
            amount, count = map(float, value.split(','))

            # If this is the first location or same location, accumulate
            if current_location == location:
                total_revenue += amount
                trip_count += count
            else:
                # If we have a previous location, store its results
                if current_location:
                    avg_revenue = total_revenue / trip_count
                    results.append((current_location, int(trip_count), total_revenue, avg_revenue))

                # Start accumulating for new location
                current_location = location
                total_revenue = amount
                trip_count = count

        except ValueError:
            continue

    # Store the last location
    if current_location:
        avg_revenue = total_revenue / trip_count
        results.append((current_location, int(trip_count), total_revenue, avg_revenue))

    # Sort results by total revenue in descending order
    results.sort(key=lambda x: x[2], reverse=True)

    # Print header
    print("LocationID\tTrip_Count\tTotal_Revenue\tAvg_Revenue")
    print("-" * 65)

    # Output sorted results
    for location, count, total_rev, avg_rev in results:
        print(f"{location}\t{count}\t${total_rev:.2f}\t${avg_rev:.2f}")

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```

The output will be stored in 3 part files sorted in descending order for each file. In order to find the pick up location having the most revenue, we need to see the first line of each part using the following command.

```
hadoop@ip-172-31-76-104:~
184     22      $1291.31        $58.70
2       25      $1290.55        $51.62
214     31      $1200.22        $38.72
206     25      $1036.96        $41.48
204     19      $1004.46        $52.87
84      12      $968.04 $80.67
245     23      $927.24 $40.31
58      17      $790.06 $46.47
176     12      $789.32 $65.78
5       12      $731.08 $60.92
109     19      $656.75 $34.57
105     28      $524.99 $18.75
187     18      $522.24 $29.01
30      11      $492.12 $44.74
46      18      $469.41 $26.08
199     21      $420.33 $20.02
44      6       $391.86 $65.31
59      13      $221.36 $17.03
27      5       $199.28 $39.86
99      5       $121.82 $24.36
110     1       $6.80   $6.80
[hadoop@ip-172-31-76-104 ~]$ hadoop fs -cat /user/hadoop/mrtask_b/part-* | head -n 7
LocationID      Trip_Count      Total_Revenue   Avg_Revenue
----------------------------------------------------------------
132     894297  $50147133.25    $56.07
138     913478  $39498237.74    $43.24
161     1452739 $21044710.01    $14.49
230     1342088 $20989352.86    $15.64
186     1363321 $19758296.33    $14.49
[hadoop@ip-172-31-76-104 ~]$
```

Answer:

Location 132 generated the most revenue of $50147133.25 with trip count of 894297.

<u>Task C</u>:

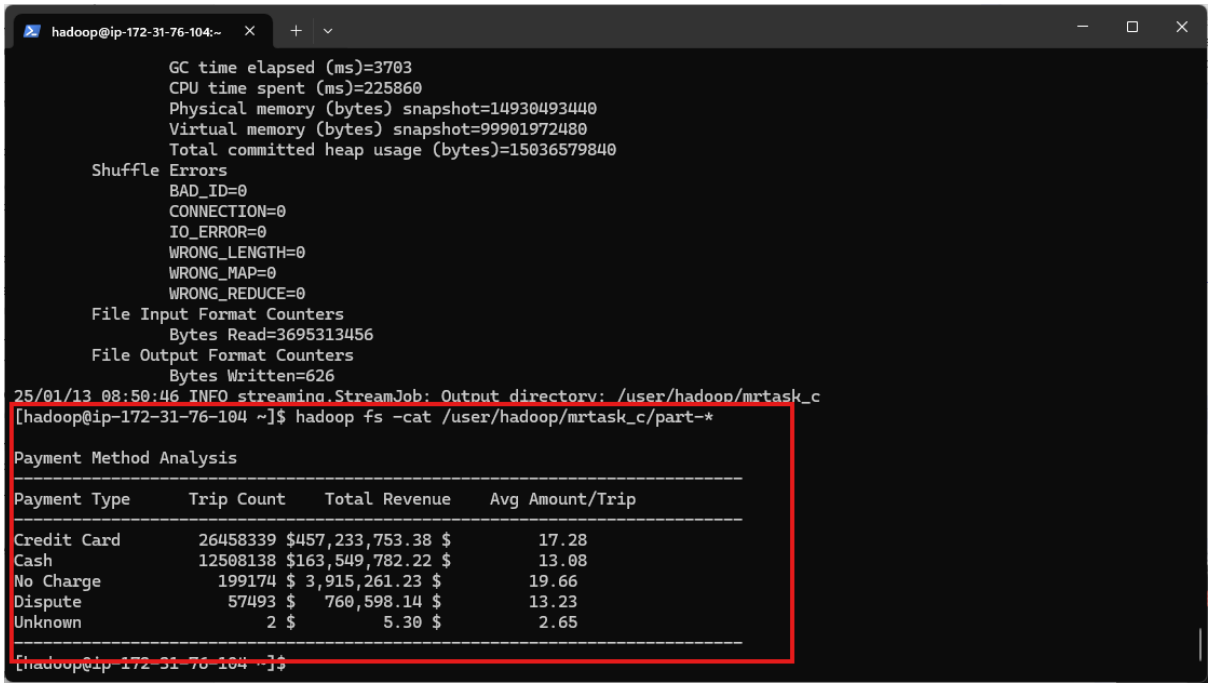
What are the different payment types used by customers and their count? The final results should be in a sorted format.

Code:

```
#!/usr/bin/env python3

import sys

def get_payment_type_name(code):
    """Convert payment type code to descriptive name"""
    payment_types = {
        '1': 'Credit Card',
        '2': 'Cash',
        '3': 'No Charge',
        '4': 'Dispute',
        '5': 'Unknown',
        '6': 'Voided Trip'
    }
    return payment_types.get(code, 'Unknown')

def mapper():
    # Skip header
    header = True

    for line in sys.stdin:
        if header:
            header = False
            continue

        # Remove leading/trailing whitespace
        line = line.strip()
```

```
    # Split the line into fields
    fields = line.split(',')

    try:
        if len(fields) >= 10:  # Ensure we have enough fields
            payment_type = fields[9]  # payment_type field
            total_amount = float(fields[16])  # total_amount for revenue calculation

            # Convert payment type code to name
            payment_name = get_payment_type_name(payment_type)

            # Emit key-value pair: payment_type \t count,amount
            print(f'{payment_name}\t1,{total_amount}')

    except ValueError:
        # Skip lines with invalid data
        continue

def reducer():
    current_payment_type = None
    payment_count = 0
    total_revenue = 0.0

    # Store results to sort later
    results = []

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        # Parse input from mapper
        payment_type, value = line.split('\t')
        count, amount = value.split(',')

        try:
            count = int(count)
            amount = float(amount)
        except ValueError:
            continue

        # If this is the first payment type or same type, accumulate
        if current_payment_type == payment_type:
            payment_count += count
            total_revenue += amount
        else:
            # If we have a previous payment type, store its results
            if current_payment_type:
                avg_amount = total_revenue / payment_count
                results.append((current_payment_type, payment_count, total_revenue, avg_amount))

            # Start accumulating for new payment type
            current_payment_type = payment_type
            payment_count = count
            total_revenue = amount

    # Store the last payment type
    if current_payment_type:
        avg_amount = total_revenue / payment_count
        results.append((current_payment_type, payment_count, total_revenue, avg_amount))

    # Sort results by count in descending order
    results.sort(key=lambda x: x[1], reverse=True)

    # Print header
    print("\nPayment Method Analysis")
    print("-" * 75)
    print("Payment Type      Trip Count    Total Revenue    Avg Amount/Trip")
    print("-" * 75)
```

```
    # Output sorted results with proper formatting
    for payment_type, count, revenue, avg in results:
        print(f"{payment_type:<16} {count:>10} ${revenue:>13,.2f} ${avg:>14,.2f}")
    print("-" * 75)

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```



Answer:

Payment Type	Trip Count	Total Revenue	Avg Amount/Trip
Credit Card	26458339	$ 457,233,753.38	$ 17.28
Cash	12508138	$ 163,549,782.22	$ 13.08
No Charge	199174	$ 3,915,261.23	$ 19.66
Dispute	57493	$ 760,598.14	$ 13.23
Unknown	2	$ 5.30	$ 2.65

Task D:

What is the average trip time for different pickup locations?

Code:

```
#!/usr/bin/env python3

import sys
from datetime import datetime

def calculate_trip_duration(pickup_str, dropoff_str):
    """Calculate trip duration in minutes"""
    try:
        pickup_time = datetime.strptime(pickup_str, '%Y-%m-%d %H:%M:%S')
        dropoff_time = datetime.strptime(dropoff_str, '%Y-%m-%d %H:%M:%S')
```

```
        duration = (dropoff_time - pickup_time).total_seconds() / 60.0
        return duration if duration >= 0 else 0
    except ValueError:
        return 0

def mapper():
    # Skip header
    header = True

    for line in sys.stdin:
        if header:
            header = False
            continue

        # Remove leading/trailing whitespace
        line = line.strip()

        # Split the line into fields
        fields = line.split(',')

        try:
            if len(fields) >= 8:  # Ensure we have enough fields
                pickup_location = fields[7]  # PULocationID
                pickup_time = fields[1]     # tpep_pickup_datetime
                dropoff_time = fields[2]    # tpep_dropoff_datetime

                # Calculate trip duration in minutes
                duration = calculate_trip_duration(pickup_time, dropoff_time)

                if duration > 0:  # Only emit valid durations
                    # Emit: pickup_location \t duration,1
                    print(f'{pickup_location}\t{duration},1')

        except (ValueError, IndexError):
            # Skip lines with invalid data
            continue

def reducer():
    current_location = None
    total_duration = 0.0
    trip_count = 0

    # Store results to sort later
    results = []

    # Print header first
    print("Location_ID\tTrip_Count\tAverage_Duration_Minutes")

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        # Parse input from mapper
        try:
            location, value = line.split('\t')
            duration, count = map(float, value.split(','))

            # If this is the first location or same location, accumulate
            if current_location == location:
                total_duration += duration
                trip_count += count
            else:
                # If we have a previous location, store its results
                if current_location:
                    avg_duration = total_duration / trip_count
                    results.append((current_location, trip_count, avg_duration))

                # Start accumulating for new location
                current_location = location
                total_duration = duration
```

```
            trip_count = count

        except ValueError:
            continue

    # Store the last location
    if current_location:
        avg_duration = total_duration / trip_count
        results.append((current_location, trip_count, avg_duration))

    # Sort results by average duration in descending order
    results.sort(key=lambda x: x[2], reverse=True)

    # Output sorted results
    for location, count, avg_duration in results:
        print(f'{location}\t{int(count)}\t{avg_duration:.2f}')

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```

Answer:

```
hadoop@ip-172-31-76-104:~

                CPU time spent (ms)=1443860
                Physical memory (bytes) snapshot=15397462016
                Virtual memory (bytes) snapshot=99950096384
                Total committed heap usage (bytes)=15647899648
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=3695313456
        File Output Format Counters
                Bytes Written=3865
25/01/13 10:11:27 INFO streaming.StreamJob: Output directory: /user/hadoop/mrtask_d
[hadoop@ip-172-31-76-104 ~]$ hadoop fs -cat /user/hadoop/mrtask_d/part-*
Location_ID     Trip_Count      Average_Duration_Minutes
109     19      79.04
9       125     65.06
139     69      59.03
187     18      48.01
10      6520    47.26
215     2430    46.68
132     893283  42.33
219     1816    41.93
2       25      37.36
30      11      35.60
138     912512  35.17
176     11      34.96
```

hadoop@ip-172-31-76-104:~

```
30      11      35.60
138     912512  35.17
176     11      34.96
130     2892    34.72
122     70      33.72
93      4361    31.48
38      99      31.08
31      125     29.84
205     138     28.80
222     69      28.12
124     373     27.94
16      194     27.04
28      2364    26.76
216     2201    26.32
101     120     25.76
102     331     25.08
3       112     24.83
154     109     24.67
208     256     24.43
12      13626   24.05
70      5477    23.91
212     505     23.00
194     2013    22.68
254     239     22.52
117     49      22.46
203     148     22.45
22      509     22.35
218     248     21.85
192     375     21.60
160     630     21.45
```

hadoop@ip-172-31-76-104:~

```
218     248     21.85
192     375     21.60
160     630     21.45
261     217688  21.34
185     286     21.16
85      1004    21.11
253     81      21.09
76      1092    21.01
35      591     20.96
88      154639  20.92
184     22      20.58
91      584     20.12
258     519     20.07
59      13      19.81
157     1784    19.76
180     252     19.74
87      295893  19.74
39      627     19.60
56      1363    19.34
55      355     19.28
11      218     19.19
195     1922    19.18
72      491     19.07
156     299     18.99
177     1034    18.95
252     233     18.80
13      361538  18.77
92      1780    18.71
155     208     18.58
150     105     18.50
```

hadoop@ip-172-31-76-104:~

```
92      1780    18.71
155     208     18.58
150     105     18.50
209     74389   18.45
149     222     18.36
135     666     18.18
47      522     18.18
217     4120    18.16
259     223     18.15
45      87700   18.10
73      106     18.06
210     247     17.84
51      210     17.78
95      5794    17.76
29      406     17.73
250     445     17.69
196     3098    17.68
33      42711   17.66
71      479     17.57
66      15723   17.56
105     28      17.48
248     316     17.47
199     21      17.45
126     439     17.40
52      23724   17.25
169     835     17.09
67      249     17.08
232     79590   17.04
14      1587    17.00
77      325     16.86
```

hadoop@ip-172-31-76-104:~

```
232     79590   17.04
14      1587    17.00
77      325     16.86
53      272     16.72
8       470     16.72
36      6898    16.71
144     452373  16.56
148     588903  16.56
186     1362468 16.54
63      1360    16.53
133     2064    16.53
65      53299   16.52
158     463834  16.46
206     26      16.44
202     1779    16.43
226     71664   16.41
231     736788  16.41
211     329999  16.40
230     1341431 16.39
244     41510   16.29
134     1851    16.27
188     5218    16.22
37      12968   16.20
62      1866    16.19
190     2001    16.17
123     636     16.15
243     8382    16.11
256     47451   16.10
21      227     16.06
84      12      16.05
```

hadoop@ip-172-31-76-104:~

```
256     47451   16.10
21      227     16.06
84      12      16.05
40      23757   15.97
83      5127    15.86
114     570597  15.77
125     241158  15.73
235     885     15.70
119     1235    15.68
89      3688    15.67
54      2122    15.64
247     7641    15.58
255     53054   15.57
213     479     15.57
181     49833   15.56
246     566377  15.53
257     1506    15.50
127     4254    15.47
165     605     15.46
131     237     15.46
61      9729    15.46
116     47987   15.44
97      39565   15.44
163     1026825 15.40
228     4291    15.35
34      1237    15.31
197     1326    15.27
46      18      15.25
96      93      15.23
161     1452000 15.18
```

hadoop@ip-172-31-76-104:~

```
46      18      15.25
96      93      15.23
161     1452000 15.18
64      89      15.14
79      1213228 15.14
43      497107  15.13
164     934736  15.12
189     13016   15.10
68      915841  15.10
100     780511  15.10
128     237     14.93
260     24783   14.93
172     25      14.92
162     1315508 14.92
25      42086   14.91
233     500625  14.89
146     42783   14.89
225     4917    14.81
4       110718  14.80
113     667915  14.79
249     854595  14.79
234     1314511 14.72
159     2348    14.71
173     1407    14.70
50      377317  14.65
193     20007   14.61
69      1960    14.60
80      24315   14.60
81      134     14.58
182     419     14.53
```

hadoop@ip-172-31-76-104:~

```
80      24315   14.60
81      134     14.58
182     419     14.53
23      54      14.52
170     1248339 14.44
264     635795  14.44
106     7599    14.40
179     25602   14.39
48      1221858 14.39
221     220     14.38
166     207392  14.36
115     218     14.36
82      11372   14.35
129     23309   14.31
121     282     14.12
227     464     14.07
112     24057   14.07
175     100     14.07
223     24864   14.05
90      749057  13.96
57      175     13.95
152     40378   13.92
107     948045  13.90
174     377     13.86
242     604     13.85
153     424     13.84
78      479     13.83
240     123     13.80
86      50      13.78
147     378     13.76
```

hadoop@ip-172-31-76-104:~

```
240     123     13.80
86      50      13.78
147     378     13.76
140     657082  13.72
49      16230   13.71
15      61      13.69
17      12319   13.64
137     500050  13.62
171     216     13.61
224     116981  13.61
108     333     13.53
120     233     13.49
142     1110895 13.46
167     1010    13.34
24      116881  13.32
20      275     13.26
168     7707    13.22
7       71535   13.19
229     740483  13.12
198     1586    13.06
42      66574   13.01
151     336845  12.96
239     997449  12.87
143     417690  12.82
41      142336  12.78
99      5       12.73
118     89      12.69
200     270     12.67
75      257872  12.66
191     272     12.65
```

```
hadoop@ip-172-31-76-104:~
200     270     12.67
75      257872  12.66
191     272     12.65
238     737732  12.65
241     422     12.61
145     46781   12.60
262     462722  12.59
32      126     12.54
236     1392013 12.46
94      267     12.44
74      142320  12.39
5       12      12.37
60      316     12.35
98      168     12.33
251     133     12.23
26      817     12.22
220     895     12.20
18      512     12.14
237     1494128 12.11
183     65      12.10
141     890370  12.06
263     688601  11.95
27      5       11.77
136     513     11.51
214     31      11.36
201     52      11.06
19      210     10.75
111     92      10.36
44      6       9.78
265     18647   9.60
```

```
hadoop@ip-172-31-76-104:~
94      267     12.44
74      142320  12.39
5       12      12.37
60      316     12.35
98      168     12.33
251     133     12.23
26      817     12.22
220     895     12.20
18      512     12.14
237     1494128 12.11
183     65      12.10
141     890370  12.06
263     688601  11.95
27      5       11.77
136     513     11.51
214     31      11.36
201     52      11.06
19      210     10.75
111     92      10.36
44      6       9.78
265     18647   9.60
207     1896    8.76
1       2600    7.71
6       168     7.03
58      18      6.42
178     577     6.23
245     22      5.45
204     20      4.75
110     1       3.18
[hadoop@ip-172-31-76-104 ~]$
```

<u>Task E</u>:

Calculate the average tips to revenue ratio of the drivers for different pickup locations in sorted format.

Code:

```
#!/usr/bin/env python3

import sys

def mapper():
```

```
    # Skip header
    header = True

    for line in sys.stdin:
        if header:
            header = False
            continue

        # Remove leading/trailing whitespace
        line = line.strip()

        # Split the line into fields
        fields = line.split(',')

        try:
            if len(fields) >= 17:  # Ensure we have enough fields
                pickup_location = fields[7]    # PULocationID
                fare_amount = float(fields[10])  # fare_amount
                tip_amount = float(fields[13])   # tip_amount

                # Only consider rides with valid fare amounts
                if fare_amount > 0:
                    # Emit: pickup_location \t tip_amount,fare_amount,1
                    print(f'{pickup_location}\t{tip_amount},{fare_amount},1')

        except (ValueError, IndexError):
            # Skip lines with invalid data
            continue

def reducer():
    current_location = None
    total_tips = 0.0
    total_fare = 0.0
    trip_count = 0

    # Store results to sort later
    results = []

    # Print header
    print("Location_ID\tTrip_Count\tTotal_Tips\tTotal_Fare\tTip_Ratio_Percentage")

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        try:
            # Parse input from mapper
            location, values = line.split('\t')
            tip, fare, count = map(float, values.split(','))

            # If this is the first location or same location, accumulate
            if current_location == location:
                total_tips += tip
                total_fare += fare
                trip_count += count
            else:
                # If we have a previous location, calculate and store its results
                if current_location:
                    if total_fare > 0:  # Avoid division by zero
                        tip_ratio = (total_tips / total_fare) * 100  # Convert to percentage
                        results.append((current_location, trip_count, total_tips, total_fare, tip_ratio))

                # Start accumulating for new location
                current_location = location
                total_tips = tip
                total_fare = fare
                trip_count = count

        except ValueError:
            continue
```

```
    # Calculate and store the last location
    if current_location and total_fare > 0:
        tip_ratio = (total_tips / total_fare) * 100
        results.append((current_location, trip_count, total_tips, total_fare, tip_ratio))

    # Sort results by tip ratio in descending order
    results.sort(key=lambda x: x[4], reverse=True)

    # Output sorted results with formatted numbers
    for location, count, tips, fare, ratio in results:
        print(f'{location}\t{int(count)}\t${tips:.2f}\t${fare:.2f}\t{ratio:.2f}%')

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```

Answer:

```
hadoop@ip-172-31-76-104:~

25/01/13 10:34:16 INFO streaming.StreamJob: Output directory: /user/hadoop/mrtask_e
[hadoop@ip-172-31-76-104 ~]$ hadoop fs -cat /user/hadoop/mrtask_e/part-*
Location_ID     Trip_Count      Total_Tips      Total_Fare      Tip_Ratio_Percentage
30      11      $88.64  $372.00 23.83%
58      17      $141.62 $632.34 22.40%
178     574     $1124.59        $5472.64        20.55%
176     11      $131.26 $641.50 20.46%
109     16      $100.35 $493.50 20.33%
172     25      $214.87 $1074.88        19.99%
117     49      $372.28 $1886.94        19.73%
175     100     $444.26 $2296.60        19.34%
122     72      $406.60 $2135.51        19.04%
199     21      $58.95  $321.50 18.34%
84      11      $140.86 $790.88 17.81%
138     913042  $5235581.40     $29908005.33    17.51%
191     264     $1106.70        $6707.06        16.50%
252     229     $1938.65        $11808.32       16.42%
86      50      $201.84 $1241.01        16.26%
210     248     $862.22 $5412.80        15.93%
201     51      $239.07 $1505.33        15.88%
52      23727   $55065.34       $347727.46      15.84%
125     241167  $456526.26      $2898140.13     15.75%
123     630     $1482.18        $9430.63        15.72%
87      296017  $764776.00      $4925203.87     15.53%
194     2003    $13030.02       $84162.40       15.48%
162     1315928 $2343384.51     $15179281.71    15.44%
16      193     $640.86 $4160.17        15.40%
107     948304  $1534709.38     $10074896.80    15.23%
249     854761  $1428601.86     $9404549.06     15.19%
234     1314897 $2201000.76     $14501261.15    15.18%
```

hadoop@ip-172-31-76-104:~

107	948304	$1534709.38	$10074896.80	15.23%
249	854761	$1428601.86	$9404549.06	15.19%
234	1314897	$2201000.76	$14501261.15	15.18%
54	2121	$4597.12	$30416.92	15.11%
246	566599	$991717.87	$6565467.84	15.11%
170	1248646	$2090359.92	$13875555.41	15.07%
40	23759	$49486.51	$328727.12	15.05%
13	361702	$877673.84	$5837151.56	15.04%
113	668089	$1076182.31	$7159808.21	15.03%
158	464049	$825121.37	$5503031.75	14.99%
79	1213448	$2054227.04	$13806549.22	14.88%
161	1452519	$2461456.67	$16543822.57	14.88%
64	88	$471.32	$3169.92	14.87%
184	22	$159.53	$1074.50	14.85%
114	570756	$965520.60	$6507133.64	14.84%
231	737046	$1388899.09	$9366985.08	14.83%
233	500789	$877428.52	$5941806.87	14.77%
245	23	$114.21	$774.93	14.74%
33	42721	$95318.45	$648494.52	14.70%
255	53069	$98951.25	$673430.96	14.69%
224	117021	$187443.43	$1295053.57	14.47%
66	15722	$33840.02	$234277.87	14.44%
68	916068	$1494573.04	$10384076.55	14.39%
88	154734	$386602.77	$2687126.84	14.39%
263	688712	$994772.99	$6916774.28	14.38%
143	417797	$609676.17	$4257455.68	14.32%
257	1504	$3407.73	$23835.00	14.30%
148	589053	$1066949.62	$7478920.33	14.27%
164	935078	$1524949.54	$10702274.40	14.25%
211	330105	$555657.08	$3907790.47	14.22%

hadoop@ip-172-31-76-104:~

148	589053	$1066949.62	$7478920.33	14.27%
164	935078	$1524949.54	$10702274.40	14.25%
211	330105	$555657.08	$3907790.47	14.22%
137	500269	$759414.07	$5342383.07	14.21%
262	462886	$696270.37	$4913750.89	14.17%
229	740640	$1101595.56	$7776949.04	14.16%
163	1027122	$1678710.06	$11852618.36	14.16%
166	207421	$347052.28	$2453536.13	14.14%
144	452516	$761621.75	$5386615.76	14.14%
90	749236	$1178744.70	$8337792.67	14.14%
141	890551	$1219976.78	$8656617.51	14.09%
142	1111322	$1649622.32	$11717610.69	14.08%
209	74424	$158666.65	$1129372.59	14.05%
1	2486	$26792.84	$190958.67	14.03%
106	7605	$13336.14	$95536.75	13.96%
264	637009	$1204704.68	$8634807.71	13.95%
181	49846	$90497.25	$649708.41	13.93%
189	13016	$22196.47	$159645.66	13.90%
237	1494586	$1938794.25	$13960711.08	13.89%
236	1392467	$1915613.83	$13838498.00	13.84%
93	4331	$26862.22	$194085.08	13.84%
238	737924	$1065195.62	$7698244.42	13.84%
140	657324	$1018628.20	$7369603.70	13.82%
195	1919	$4467.42	$32570.07	13.72%
111	92	$187.28	$1369.51	13.67%
186	1363124	$2170943.52	$15889549.23	13.66%
239	997783	$1456767.60	$10676022.65	13.65%
63	1358	$2791.61	$20510.40	13.61%
80	24302	$41190.40	$302667.35	13.61%
50	377420	$589174.84	$4335885.49	13.59%

hadoop@ip-172-31-76-104:~

63	1358	$2791.61	$20510.40	13.61%
80	24302	$41190.40	$302667.35	13.61%
50	377420	$589174.84	$4335885.49	13.59%
230	1341823	$2261560.16	$16681828.71	13.56%
112	24095	$41218.69	$304235.49	13.55%
15	61	$262.83	$1942.15	13.53%
8	468	$1724.40	$12812.83	13.46%
215	2397	$16883.03	$125787.72	13.42%
256	47443	$80036.27	$598195.16	13.38%
10	6490	$45176.16	$338456.90	13.35%
71	479	$1034.56	$7751.18	13.35%
25	42104	$72682.08	$545450.93	13.33%
151	336983	$474480.42	$3563593.78	13.31%
48	1221997	$1810436.99	$13648300.69	13.26%
36	6888	$11827.02	$89428.72	13.23%
258	511	$1411.03	$10680.40	13.21%
4	110726	$172524.56	$1306561.96	13.20%
253	78	$490.66	$3731.01	13.15%
24	116925	$169325.69	$1290628.19	13.12%
261	217816	$461145.24	$3519401.04	13.10%
43	497232	$694320.07	$5323611.49	13.04%
77	324	$809.93	$6230.51	13.00%
70	5447	$20576.63	$159458.05	12.90%
265	17874	$164154.08	$1277139.79	12.85%
115	218	$394.57	$3080.58	12.81%
100	780741	$1118605.33	$8794692.71	12.72%
132	893454	$5245584.56	$41311304.55	12.70%
23	54	$370.02	$2918.07	12.68%
180	253	$872.16	$6892.01	12.65%
65	53312	$98420.07	$779894.84	12.62%

hadoop@ip-172-31-76-104:~

23	54	$370.02	$2918.07	12.68%
180	253	$872.16	$6892.01	12.65%
65	53312	$98420.07	$779894.84	12.62%
34	1238	$2058.02	$16353.30	12.58%
134	1845	$5594.06	$44509.32	12.57%
130	2868	$15034.55	$119977.07	12.53%
216	2168	$11755.35	$94178.59	12.48%
2	25	$131.41	$1054.00	12.47%
49	16218	$23698.86	$191421.77	12.38%
96	93	$196.71	$1589.50	12.38%
190	2001	$3392.03	$27535.05	12.32%
67	248	$462.93	$3772.88	12.27%
157	1780	$6005.37	$49388.21	12.16%
76	1089	$2859.64	$23607.24	12.11%
223	24821	$44378.30	$366518.80	12.11%
9	125	$376.86	$3112.66	12.11%
251	133	$299.51	$2477.26	12.09%
28	2346	$10516.39	$87005.02	12.09%
150	102	$450.13	$3726.63	12.08%
45	87727	$142749.78	$1182722.58	12.07%
135	667	$1361.24	$11281.10	12.07%
56	1357	$4043.08	$33617.79	12.03%
139	67	$237.34	$1981.83	11.98%
128	237	$489.85	$4091.50	11.97%
97	39573	$59958.72	$500847.87	11.97%
121	281	$742.33	$6210.07	11.95%
37	12961	$18888.30	$158171.48	11.94%
219	1792	$11222.14	$94423.02	11.88%
228	4286	$7363.04	$61953.20	11.88%
98	161	$347.94	$2934.86	11.86%

hadoop@ip-172-31-76-104:~

```
219     1792    $11222.14       $94423.02       11.88%
228     4286    $7363.04        $61953.20       11.88%
98      161     $347.94 $2934.86        11.86%
213     478     $1237.96        $10483.77       11.81%
200     268     $622.35 $5328.52        11.68%
145     46888   $76806.75       $658856.88      11.66%
101     121     $453.41 $3899.90        11.63%
221     217     $450.21 $3879.82        11.60%
202     1781    $3298.06        $28516.70       11.57%
133     2061    $3598.56        $31263.63       11.51%
160     628     $1492.19        $12967.06       11.51%
243     8372    $14471.87       $127415.04      11.36%
244     41491   $70255.67       $624005.81      11.26%
35      588     $1130.07        $10038.54       11.26%
61      9717    $14317.92       $127875.48      11.20%
57      176     $399.19 $3565.68        11.20%
118     88      $179.90 $1621.80        11.09%
203     143     $697.59 $6341.17        11.00%
89      3681    $6178.75        $56239.28       10.99%
155     202     $661.00 $6018.63        10.98%
41      142327  $162499.85      $1480185.93     10.98%
108     330     $693.81 $6327.86        10.96%
222     70      $341.73 $3121.00        10.95%
154     106     $459.69 $4202.10        10.94%
75      257890  $294498.57      $2693886.57     10.93%
116     47962   $67578.05       $623691.34      10.84%
187     17      $44.10  $407.70 10.82%
55      355     $775.08 $7171.40        10.81%
73      106     $187.00 $1750.10        10.69%
198     1580    $2358.65        $22128.58       10.66%
```

hadoop@ip-172-31-76-104:~

```
55      355     $775.08 $7171.40        10.81%
73      106     $187.00 $1750.10        10.69%
198     1580    $2358.65        $22128.58       10.66%
102     329     $602.58 $5688.55        10.59%
14      1586    $3120.47        $29508.89       10.57%
124     370     $1790.08        $16978.70       10.54%
146     42770   $55976.89       $533057.36      10.50%
21      229     $562.17 $5361.00        10.49%
51      206     $571.44 $5471.22        10.44%
197     1325    $3561.78        $34536.76       10.31%
225     4926    $6254.59        $60681.07       10.31%
127     4253    $6708.57        $65110.18       10.30%
156     297     $563.23 $5471.32        10.29%
196     3093    $5916.28        $57600.98       10.27%
12      13646   $23708.06       $233286.28      10.16%
226     71629   $106056.96      $1043663.12     10.16%
179     25603   $31853.15       $313606.00      10.16%
95      5782    $9774.57        $96460.37       10.13%
27      5       $17.70  $174.92 10.12%
17      12307   $14134.82       $140220.43      10.08%
227     464     $754.80 $7513.60        10.05%
205     134     $337.85 $3373.68        10.01%
212     502     $778.53 $7806.94        9.97%
152     40287   $46976.95       $476009.03      9.87%
81      133     $256.36 $2694.65        9.51%
131     232     $501.51 $5300.85        9.46%
171     216     $363.00 $3846.73        9.44%
72      489     $798.89 $8495.35        9.40%
7       71343   $76474.04       $815472.59      9.38%
259     219     $474.41 $5095.84        9.31%
```

hadoop@ip-172-31-76-104:~

```
72      489     $798.89 $8495.35        9.40%
7       71343   $76474.04       $815472.59      9.38%
259     219     $474.41 $5095.84        9.31%
62      1858    $2298.05        $24978.41       9.20%
217     4114    $3946.74        $42941.06       9.19%
220     892     $1298.83        $14235.58       9.12%
206     25      $77.56  $851.42 9.11%
214     30      $92.31  $1024.51        9.01%
149     221     $402.90 $4472.32        9.01%
120     231     $501.17 $5577.22        8.99%
188     5204    $6250.36        $70874.20       8.82%
208     252     $700.21 $7972.75        8.78%
232     79598   $140150.60      $1617064.64     8.67%
177     1028    $1531.28        $17689.05       8.66%
91      582     $1009.80        $11665.63       8.66%
39      624     $1266.81        $14824.55       8.55%
247     7636    $8504.08        $100585.90      8.45%
74      142250  $126076.12      $1491467.49     8.45%
42      66533   $59232.28       $703509.05      8.42%
218     247     $538.43 $6397.62        8.42%
167     1006    $1093.39        $13040.72       8.38%
53      275     $425.54 $5100.00        8.34%
20      273     $368.62 $4451.40        8.28%
26      818     $912.54 $11106.93       8.22%
260     24743   $27816.89       $339485.17      8.19%
126     439     $623.42 $7629.53        8.17%
192     374     $562.62 $6930.50        8.12%
165     601     $799.40 $9959.56        8.03%
85      1000    $1187.16        $15008.90       7.91%
185     280     $322.82 $4087.51        7.90%
```

hadoop@ip-172-31-76-104:~

```
165     601     $799.40 $9959.56        8.03%
85      1000    $1187.16        $15008.90       7.91%
185     280     $322.82 $4087.51        7.90%
207     1694    $1173.78        $15091.97       7.78%
11      219     $532.83 $6856.39        7.77%
82      11361   $12434.91       $161301.65      7.71%
38      95      $277.17 $3659.17        7.57%
105     28      $33.78  $448.00 7.54%
92      1784    $2361.55        $31324.82       7.54%
250     440     $546.93 $7265.46        7.53%
119     1231    $1274.47        $17093.01       7.46%
129     23288   $22050.23       $303157.09      7.27%
19      208     $361.50 $4984.01        7.25%
44      6       $26.05  $361.01 7.22%
6       167     $245.09 $3396.59        7.22%
31      124     $210.12 $2924.71        7.18%
242     606     $613.09 $8587.52        7.14%
241     421     $510.15 $7158.14        7.13%
173     1399    $1287.87        $18273.27       7.05%
99      5       $7.32   $106.50 6.87%
22      508     $876.07 $12934.99       6.77%
183     64      $79.75  $1185.50        6.73%
169     830     $811.79 $12105.34       6.71%
3       111     $138.21 $2070.85        6.67%
29      403     $698.95 $10510.86       6.65%
83      5121    $4453.63        $67166.51       6.63%
240     120     $199.94 $3053.50        6.55%
193     18576   $12676.90       $196124.12      6.46%
46      18      $27.01  $420.50 6.42%
174     377     $439.54 $6877.94        6.39%
```

```
hadoop@ip-172-31-76-104:~
183     64      $79.75  $1185.50        6.73%
169     830     $811.79 $12105.34       6.71%
3       111     $138.21 $2070.85        6.67%
29      403     $698.95 $10510.86       6.65%
83      5121    $4453.63        $67166.51       6.63%
240     120     $199.94 $3053.50        6.55%
193     18576   $12676.90       $196124.12      6.46%
46      18      $27.01  $420.50 6.42%
174     377     $439.54 $6877.94        6.39%
153     425     $337.86 $5426.80        6.23%
78      479     $606.28 $9769.96        6.21%
248     312     $311.00 $5099.80        6.10%
69      1945    $1380.87        $24326.77       5.68%
18      506     $401.70 $7106.43        5.65%
182     415     $379.13 $6757.31        5.61%
168     7713    $5733.06        $105705.89      5.42%
32      124     $107.38 $1989.50        5.40%
159     2347    $1477.81        $27999.40       5.28%
147     378     $320.31 $6151.50        5.21%
60      313     $212.15 $4274.02        4.96%
235     868     $651.97 $13376.56       4.87%
136     508     $334.94 $7170.04        4.67%
254     239     $204.50 $4690.30        4.36%
5       12      $28.62  $664.50 4.31%
204     19      $39.92  $940.84 4.24%
47      524     $299.84 $7155.52        4.19%
94      266     $138.30 $4203.43        3.29%
59      13      $2.00   $206.01 0.97%
110     1       $0.00   $5.50   0.00%
[hadoop@ip-172-31-76-104 ~]$
```

Task F:

How does revenue vary over time? Calculate the average trip revenue per month - analysing it by hour of the day (day vs night) and the day of the week (weekday vs weekend).

Code

```
#!/usr/bin/env python3

import sys
from datetime import datetime

def get_time_category(hour):
    """Categorize hour as day (6AM-6PM) or night (6PM-6AM)"""
    return 'day' if 6 <= hour < 18 else 'night'

def is_weekend(day_of_week):
    """Check if day is weekend (Saturday=5, Sunday=6)"""
    return day_of_week >= 5

def mapper():
    # Skip header
    header = True

    for line in sys.stdin:
        if header:
            header = False
            continue

        # Remove leading/trailing whitespace
        line = line.strip()

        # Split the line into fields
        fields = line.split(',')

        try:
            if len(fields) >= 17:  # Ensure we have enough fields
```

```
            pickup_time_str = fields[1]  # tpep_pickup_datetime
            total_amount = float(fields[16])  # total_amount

            # Parse datetime
            pickup_time = datetime.strptime(pickup_time_str, '%Y-%m-%d %H:%M:%S')

            # Extract time components
            month_year = pickup_time.strftime('%Y-%m')
            hour = pickup_time.hour
            day_of_week = pickup_time.weekday()  # Monday=0, Sunday=6

            # Categorize time periods
            time_of_day = get_time_category(hour)
            day_type = 'weekend' if is_weekend(day_of_week) else 'weekday'

            # Create composite key: month_year,time_of_day,day_type
            composite_key = f'{month_year},{time_of_day},{day_type}'

            # Emit: composite_key \t amount,1
            print(f'{composite_key}\t{total_amount},1')

        except (ValueError, IndexError):
            # Skip lines with invalid data
            continue

def reducer():
    current_key = None
    total_revenue = 0.0
    trip_count = 0

    # Store results to sort later
    results = []

    for line in sys.stdin:
        # Remove leading/trailing whitespace
        line = line.strip()

        try:
            # Parse input from mapper
            key, value = line.split('\t')
            revenue, count = map(float, value.split(','))

            # If this is the first key or same key, accumulate
            if current_key == key:
                total_revenue += revenue
                trip_count += count
            else:
                # If we have a previous key, calculate and store its results
                if current_key:
                    avg_revenue = total_revenue / trip_count
                    month, time_category, day_type = current_key.split(',')
                    results.append((month, time_category, day_type, trip_count, avg_revenue))

                # Start accumulating for new key
                current_key = key
                total_revenue = revenue
                trip_count = count

        except ValueError:
            continue

    # Calculate and store the last key
    if current_key:
        avg_revenue = total_revenue / trip_count
        month, time_category, day_type = current_key.split(',')
        results.append((month, time_category, day_type, trip_count, avg_revenue))

    # Sort results by month, then time_category, then day_type
    results.sort(key=lambda x: (x[0], x[1], x[2]))
```

```
    # Output sorted results with formatted header
    print("Month\tTime\tDay Type\tTrip Count\tAvg Revenue")
    print("-" * 60)
    for month, time_cat, day_type, count, avg_rev in results:
        print(f"{month}\t{time_cat}\t{day_type}\t{int(count)}\t${avg_rev:.2f}")

if __name__ == "__main__":
    if sys.argv[1] == "mapper":
        mapper()
    elif sys.argv[1] == "reducer":
        reducer()
```

```
hadoop@ip-172-31-76-104:~

                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=3695313456
        File Output Format Counters
                Bytes Written=681
25/01/13 10:53:02 INFO streaming.StreamJob: Output directory: /user/hadoop/mrtask_f
[hadoop@ip-172-31-76-104 ~]$ hadoop fs -cat /user/hadoop/mrtask_f/part-*
Month   Time    Day Type        Trip Count      Avg Revenue
------------------------------------------------------------
2017-01 day     weekday 4057043 $15.44
2017-01 day     weekend 1398923 $14.40
2017-01 night   weekday 2830563 $16.04
2017-01 night   weekend 1424285 $15.87
2017-02 day     weekday 3717425 $15.68
2017-02 day     weekend 1324513 $14.47
2017-02 night   weekday 2767872 $16.18
2017-02 night   weekend 1359959 $15.52
2017-03 day     weekday 4258172 $16.42
2017-03 day     weekend 1396213 $14.90
2017-03 night   weekday 3252118 $16.70
2017-03 night   weekend 1388931 $15.85
2017-04 day     weekday 3879798 $16.34
2017-04 day     weekend 1651277 $15.47
2017-04 night   weekday 2868553 $16.84
2017-04 night   weekend 1647501 $16.43
[hadoop@ip-172-31-76-104 ~]$
```

Answer

Based on the results obtained, here is the analytical summary.

- Weekday daytime consistently shows the highest trip volumes across all months, ranging from 3.7M to 4.2M trips.
- Weekend trip volumes are remarkably consistent across months, hovering around 1.3-1.4M for day trips and 1.3-1.6M for night trips.
- March stands out as the peak month for weekday daytime activity with 4,258,172 trips, suggesting possible seasonal patterns.
- Night trips consistently command higher average revenue than day trips, regardless of whether it's a weekday or weekend.
- Weekend daytime trips show the lowest average revenue throughout all months, indicating a clear pricing pattern based on demand.
- There's a gradual increase in average revenue from January to April, suggesting a possible seasonal pricing trend.

- Weekday night revenue shows the most stability in pricing, staying within the $16.04-$16.84 range.
- April shows the highest night rates for both weekday ($16.84) and weekend ($16.43), indicating a possible spring price adjustment.
- The gap between weekday and weekend rates is most pronounced during daytime hours, showing consistent price differentiation.
- The difference between night and day rates is smaller on weekends compared to weekdays, suggesting more balanced demand during weekend hours.